Università di Genova

Embedded Systems

Oscillator & Timers

Prof. Enrico Simetti

Università di Genova

Oscillators

Electronic Oscillators

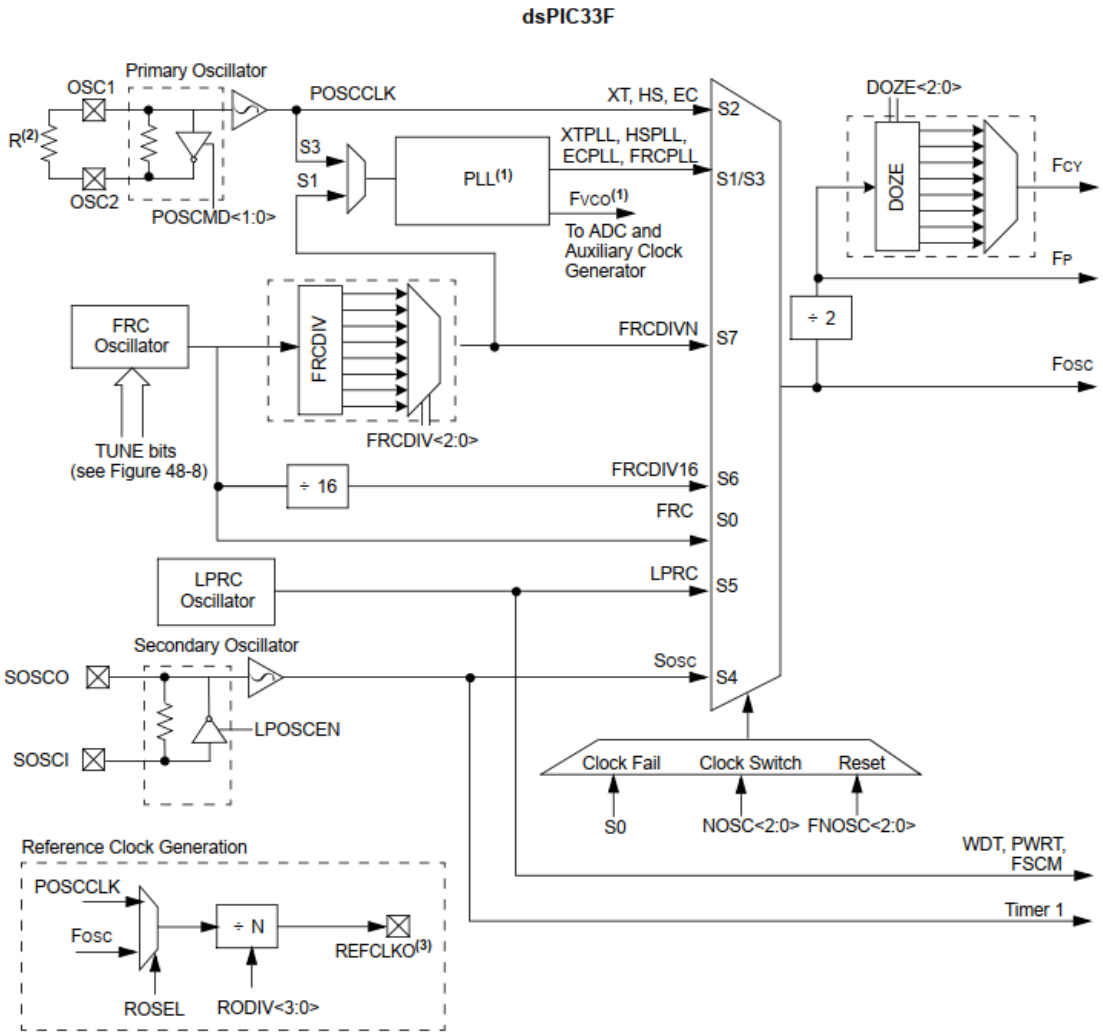
An electronic oscillator is an electronic circuit that produces a periodic, oscillating signal, powered by a DC source.

The most common is the crystal oscillator, in which the output frequency is controlled by a piezo-electric resonator (vibrating quartz crystal)



Crystal oscillator

Oscillator Circuit in dsPIC33F



Oscillator Circuit in dsPIC33F

4 clock sources available:

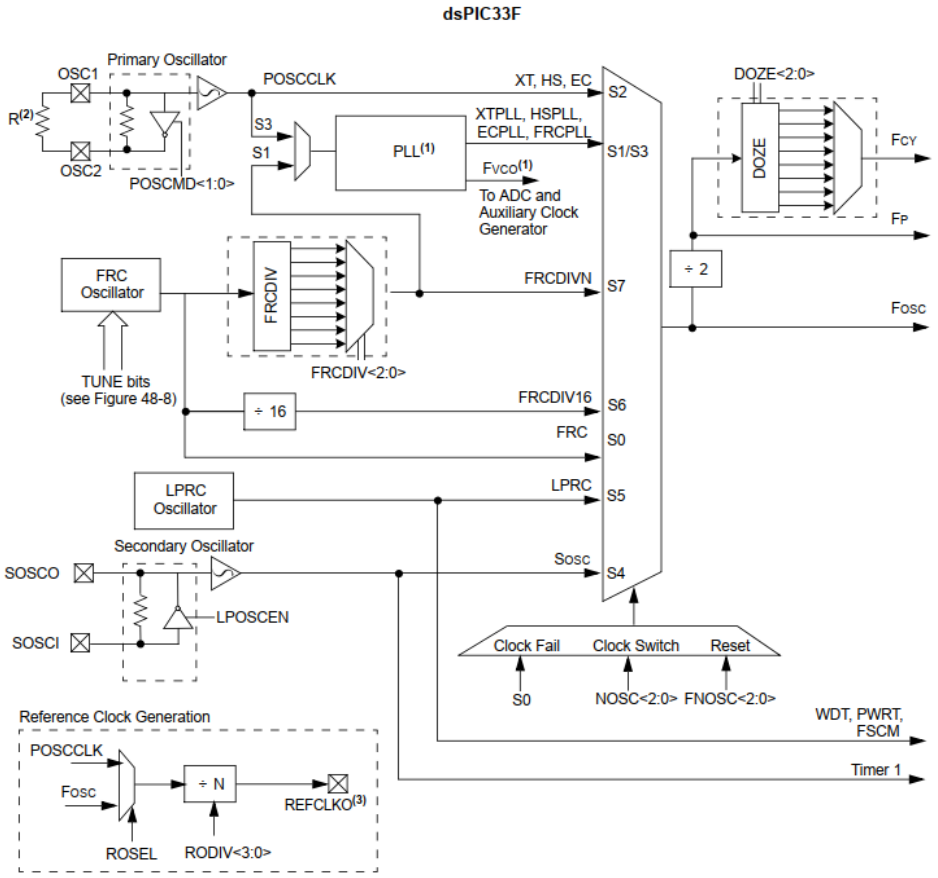
- external primary oscillator
- external secondary oscillator
- internal fast RC oscillator
- internal low-power RC oscillator

On-chip Phase-Locked Loop (PLL) (for increasing clock rate)

Clock switching between clock sources

Programmable clock postscaler (for system power savings)

A Fail-Safe Clock Monitor (FSCM) (detects clock failures)



Oscillator Source Configuration

Table 48-1: Configuration Bit Values for Clock Selection

Oscillator Source	Oscillator Mode	FNOSC Value	POSCMD Value	See Note
S0	Fast RC Oscillator (FRC)	000	xx	1
S1	Fast RC Oscillator with PLL (FRCPLL)	001	xx	1
S2	Primary Oscillator (EC)	010	00	1
S2	Primary Oscillator (XT)	010	01	—
S2	Primary Oscillator (HS)	010	10	—
S3	Primary Oscillator with PLL (ECPLL)	011	00	1
S3	Primary Oscillator with PLL (XTPLL)	011	01	—
S3	Primary Oscillator with PLL (HSPLL)	011	10	—
S4	Secondary Oscillator (SOSC)	100	xx	1
S5	Low-Power RC Oscillator	101	xx	1
S6	Fast RC Oscillator with Divide-by-16 divider (FRCDIV16)	110	xx	1
S7	Fast RC Oscillator with Divide-by-N divider (FRCDIVN)	111	xx	1, 2

Note 1: The OSC2 pin function is determined by the OSCIOFNC Configuration bit.

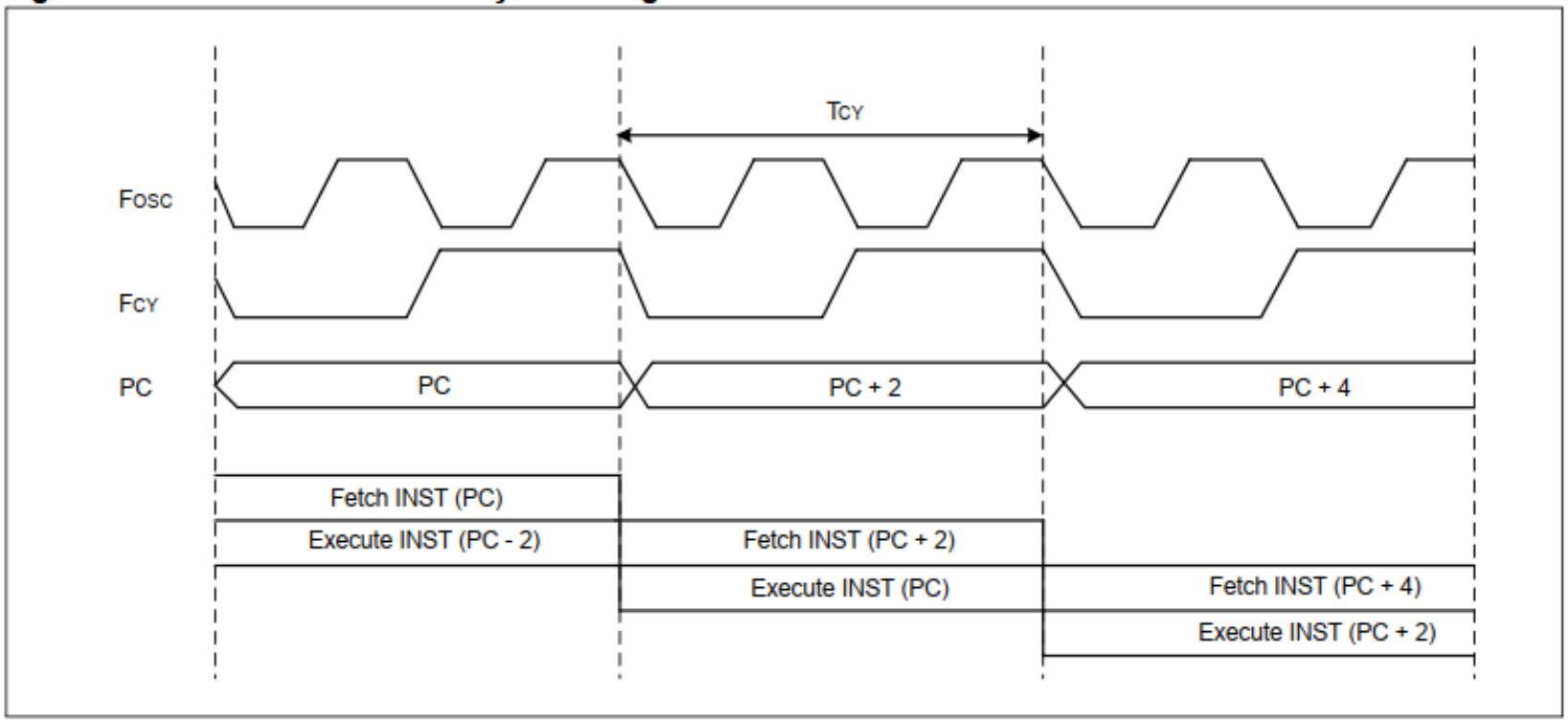
2: The default oscillator mode for an unprogrammed (erased) device.

MikroE dsPIC33F board has an XT oscillator of 8 MHz

We will use the configuration with PLL

Fosc and Fcy

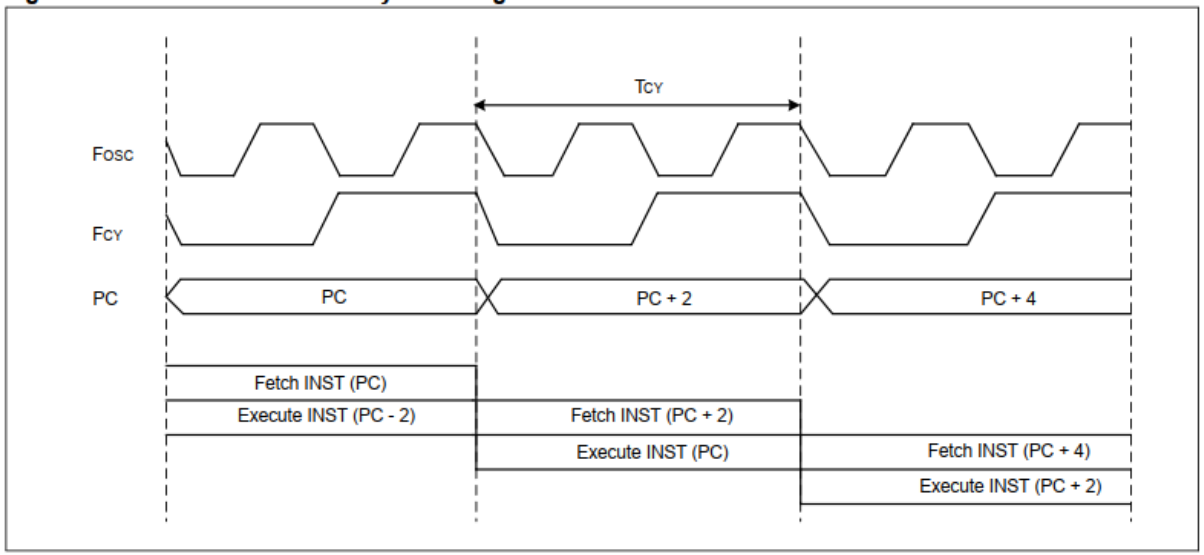
Figure 48-2: Clock/Instruction Cycle Timing



2 periods of the Fosc signal to execute one instruction

Fosc and Fcy

Figure 48-2: Clock/Instruction Cycle Timing



$$F_{cy} = \frac{F_{osc}}{2}$$

Fosc and Fcy

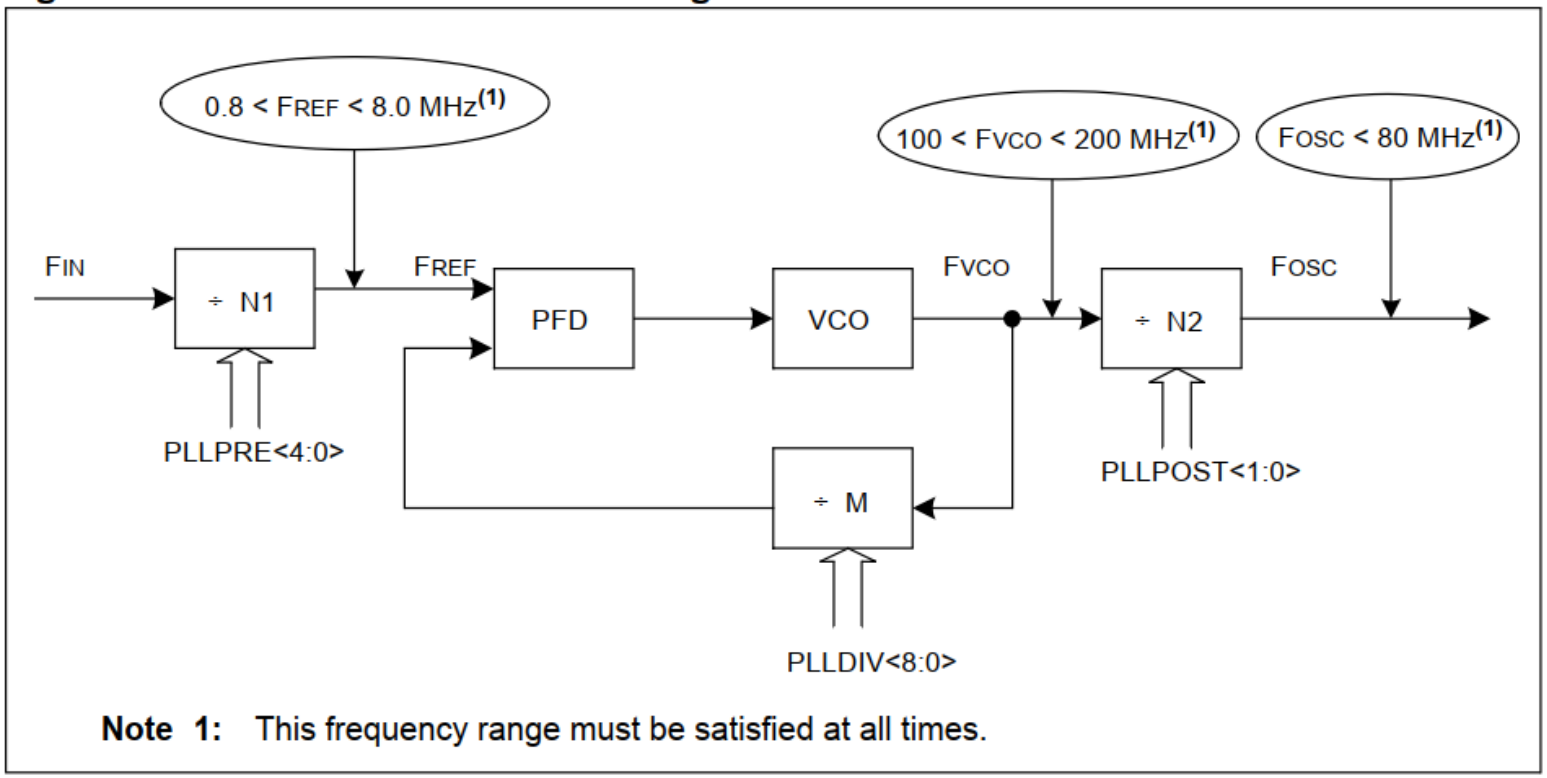
This relation is extremely important: we often need to know the value of Fcy to setup the peripherals!

Note that other devices might have a different relationship (e.g. for the dsPIC30 it is / 4)

$$F_{cy} = \frac{F_{osc}}{2}$$

Fosc and Fin

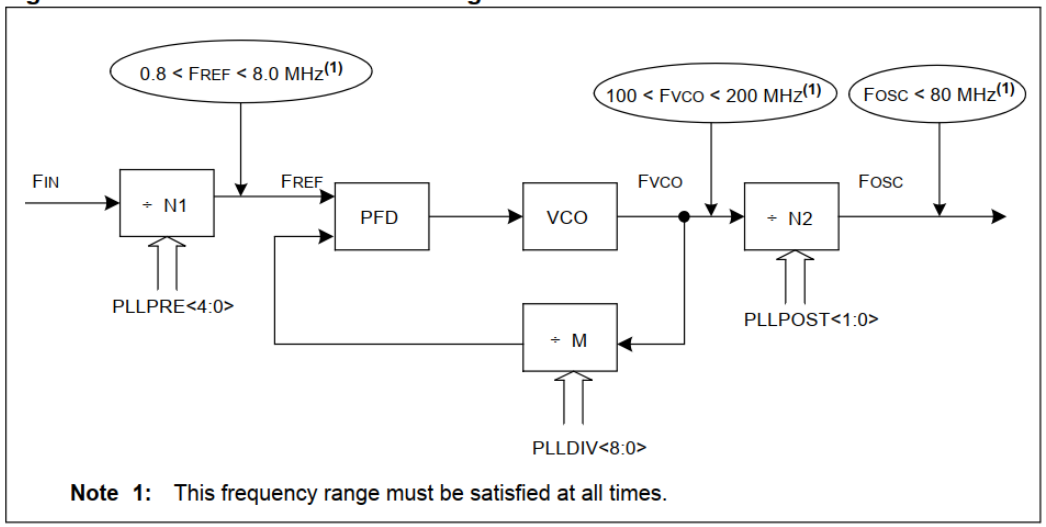
Figure 48-9: dsPIC33F PLL Block Diagram



Note 1: This frequency range must be satisfied at all times.

Fosc Calculation

Figure 48-9: dsPIC33F PLL Block Diagram



Note 1: This frequency range must be satisfied at all times.

Equation 48-3: Fosc Calculation

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2)\times 2(PLLPOST+1)}\right)$$

where:

$N1 = PLLPRE + 2$

$N2 = 2 \times (PLLPOST + 1)$

$M = PLLDIV + 2$

***FIN** is the frequency of the source before entering the PLL (8 MHz for the XT source)*

Clicker 2 Configuration

In our device, the configuration is set by the booloader and cannot be changed. It is the following one

```
PLLFBD = 70; // M = 72
CLKDIVbits.PLLPOST = 0x00; // N1
= 2
CLKDIVbits.PLLPRE = 0x00; // N2 =
2
```

Do not put these lines in the main code, they are set by the bootloader software

From Fin to Fcy

Hence with $F_{in} = 8$ MHz it results

$$F_{osc} = \frac{F_{in} M}{N_1 N_2} = 8 * \frac{72}{4} = 144 \text{ MHz}$$

Consequently

$$F_{cy} = \frac{F_{osc}}{2} = 72 \text{ MHz}$$

Università di Genova

Timers

Timers

9 16-bit timers available:

- Type A (T1) – can be operated also by the LPRC – can work in asynchronous counter mode
- Type B (T2, T4, T6, T8) – can be concatenated with C-type leading to a 32-bit timer
- Type C (T3, T5, T7, T9) – T3 and T5 can trigger an A/D conversion

functions performed:

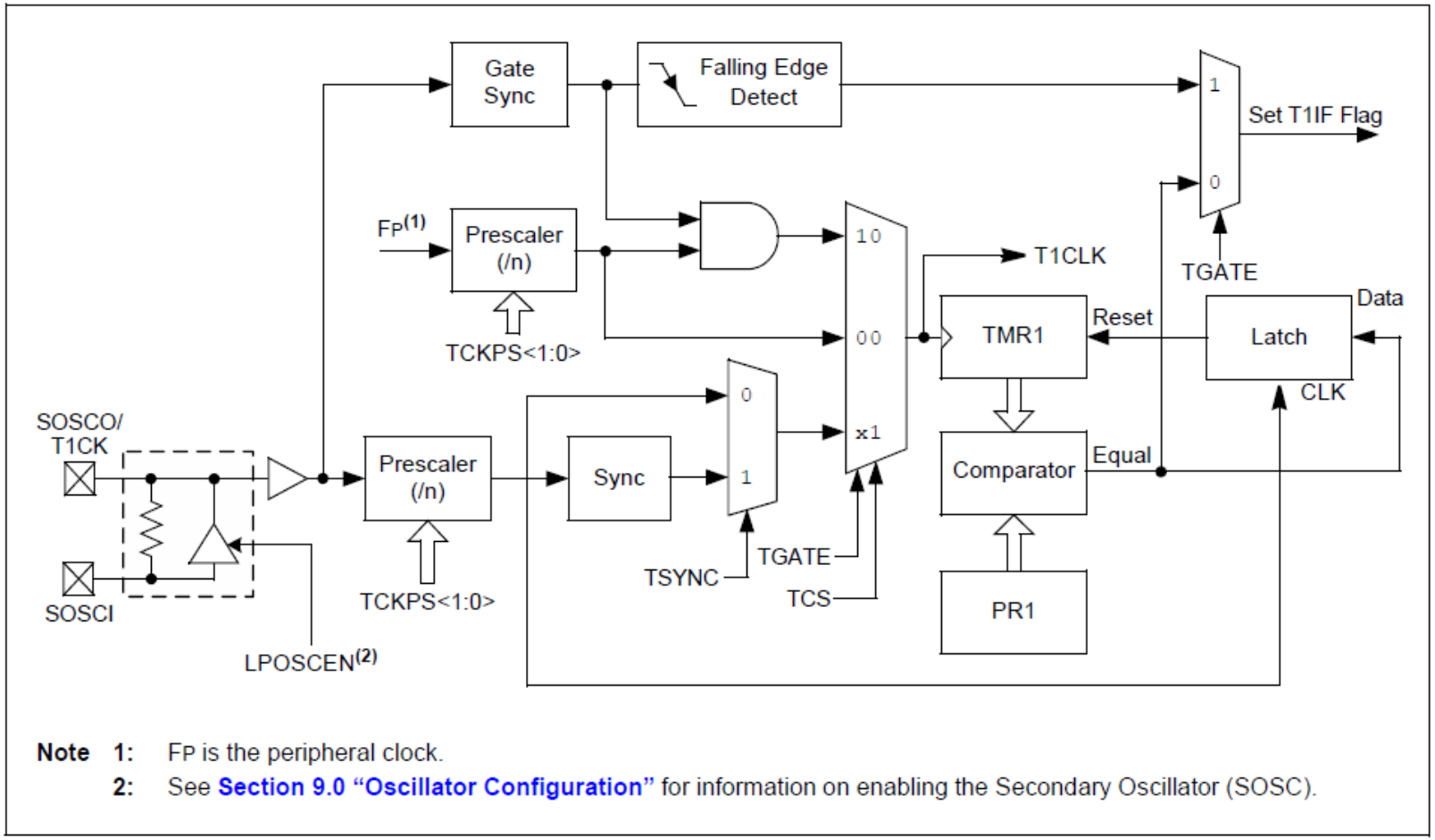
- synchronous timer – increments at every internal clock cycle (A,B,C)
- synchronous counter – increments at every external clock cycle (A,B,C)
- asynchronous counter – increments at every external raising signal (A)
- gated timer – counts the number of clock cycles an external signal is “high” (A,B,C)

Special Function Registers used for configuration and access

- TxCON, PRx, TMRx

Timer Block Diagram

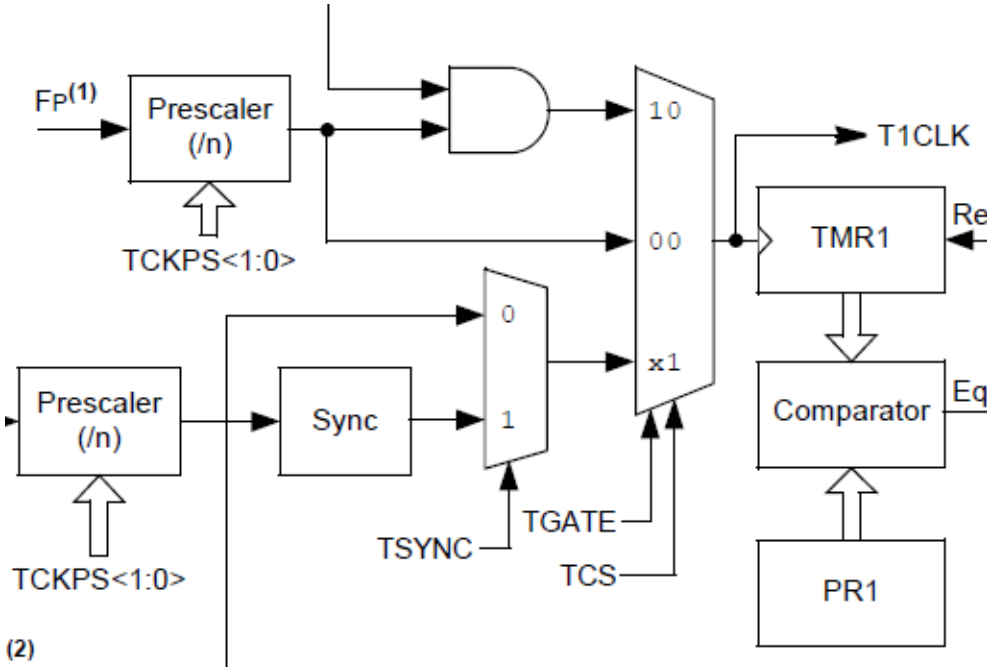
FIGURE 12-1: 16-BIT TIMER1 MODULE BLOCK DIAGRAM



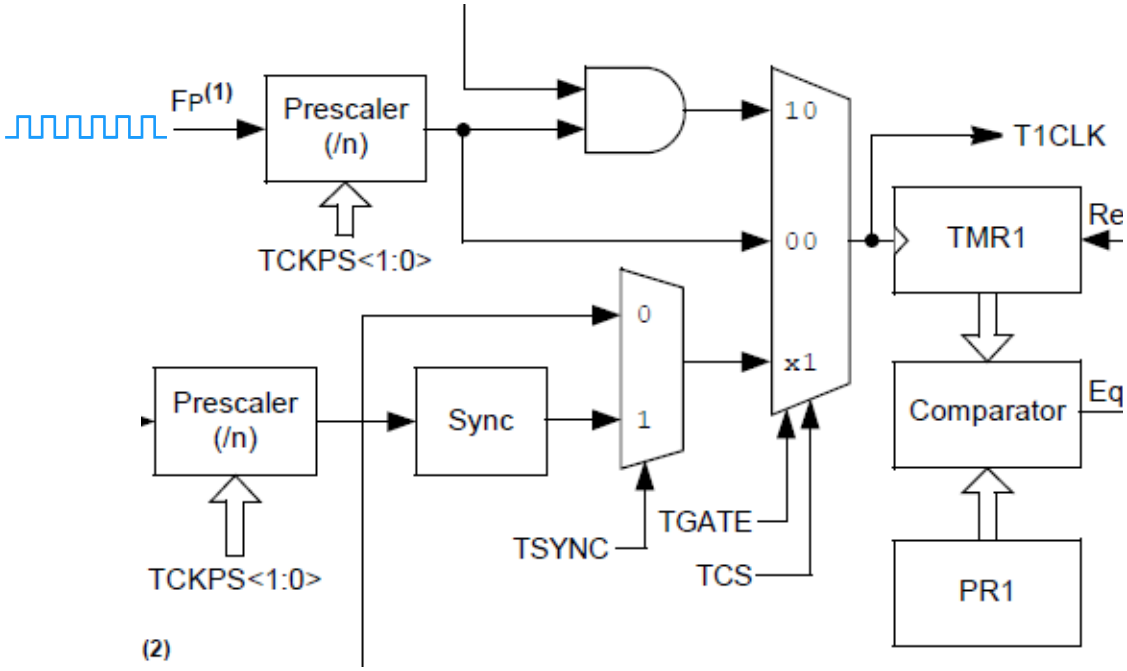
Note 1: FP is the peripheral clock.

2: See **Section 9.0 "Oscillator Configuration"** for information on enabling the Secondary Oscillator (SOSC).

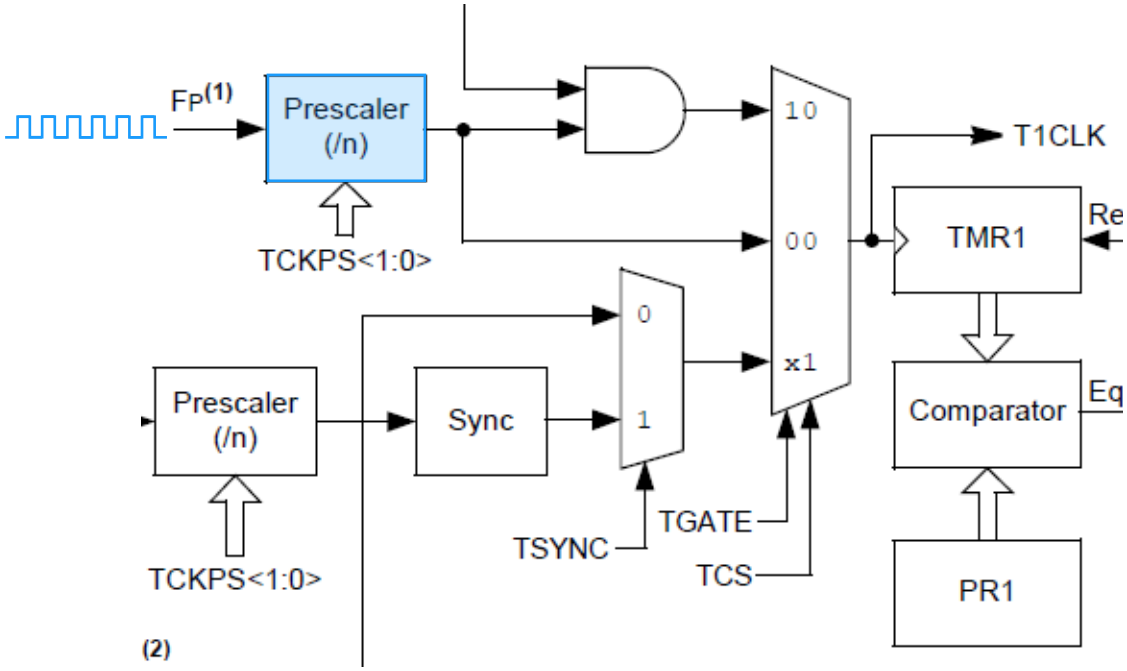
Timer Block Diagram



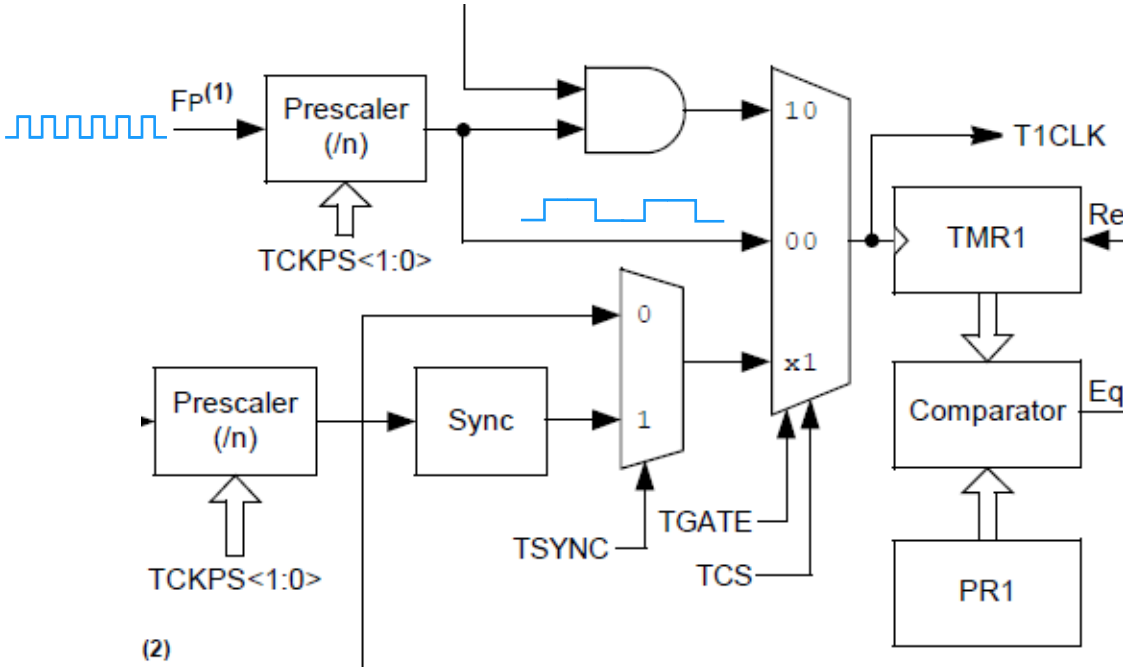
Timer Block Diagram



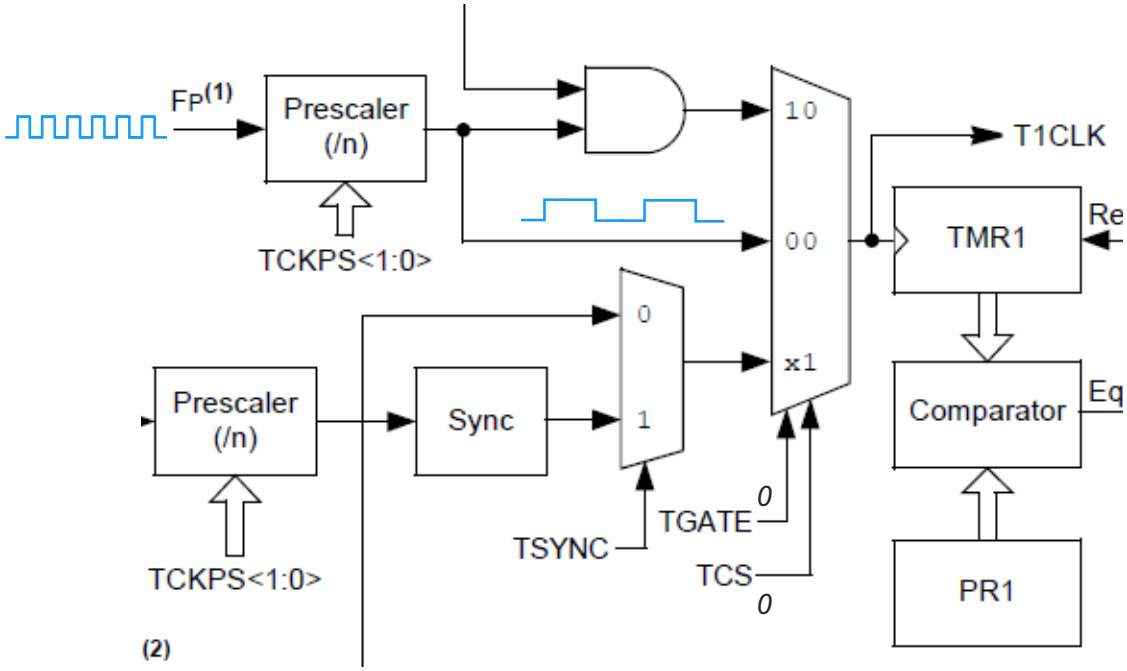
Timer Block Diagram



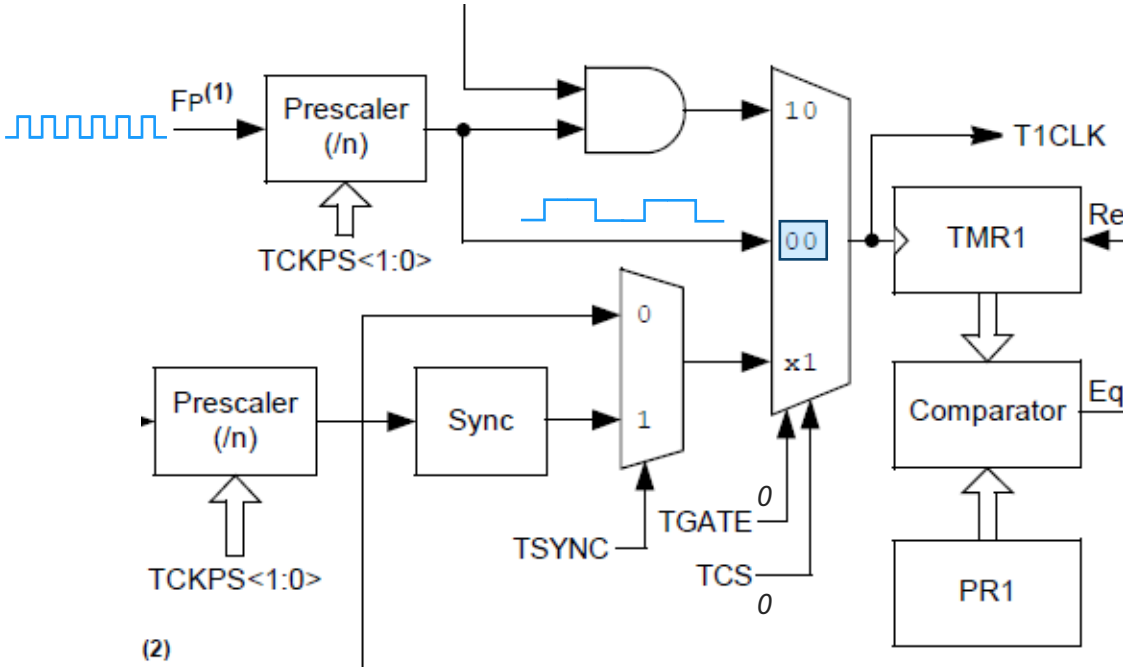
Timer Block Diagram



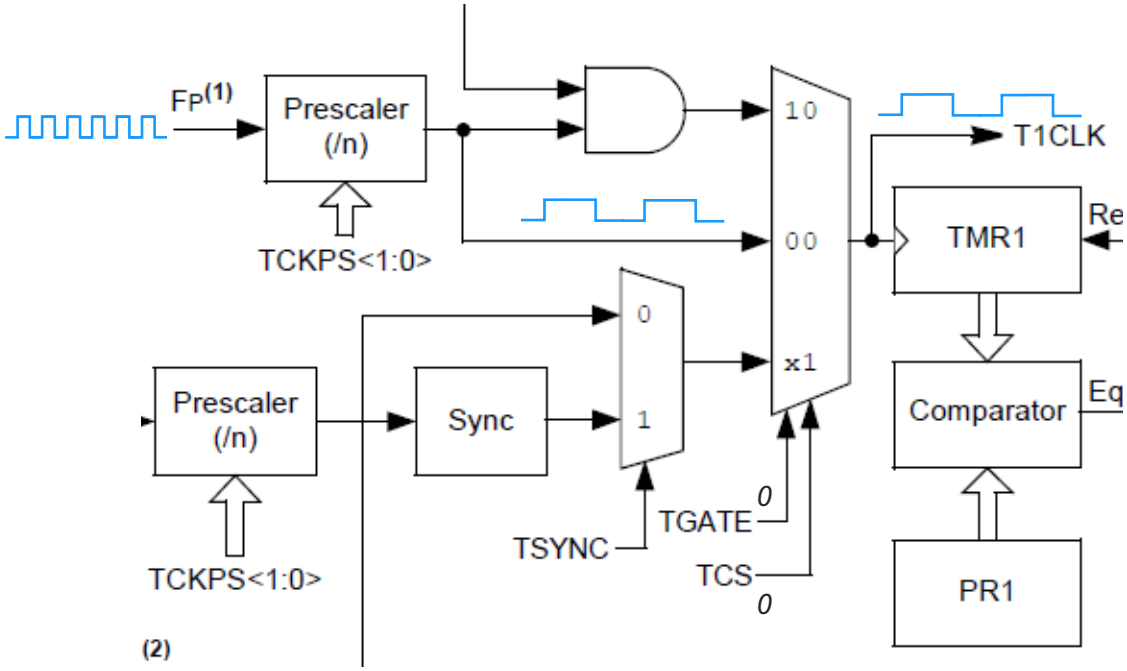
Timer Block Diagram



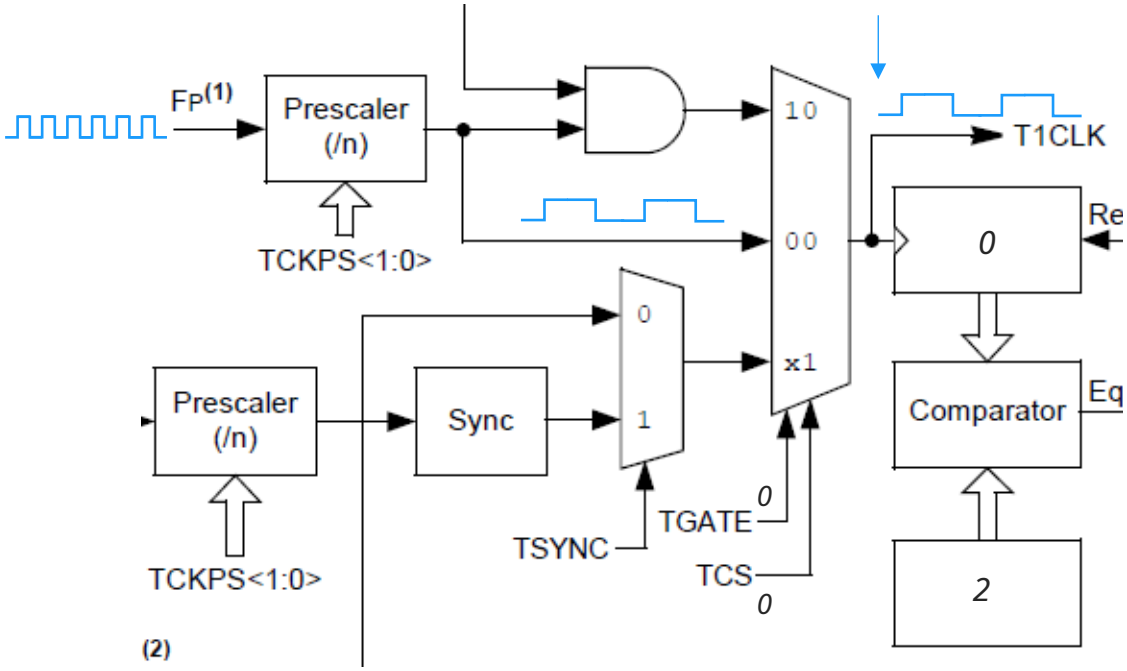
Timer Block Diagram



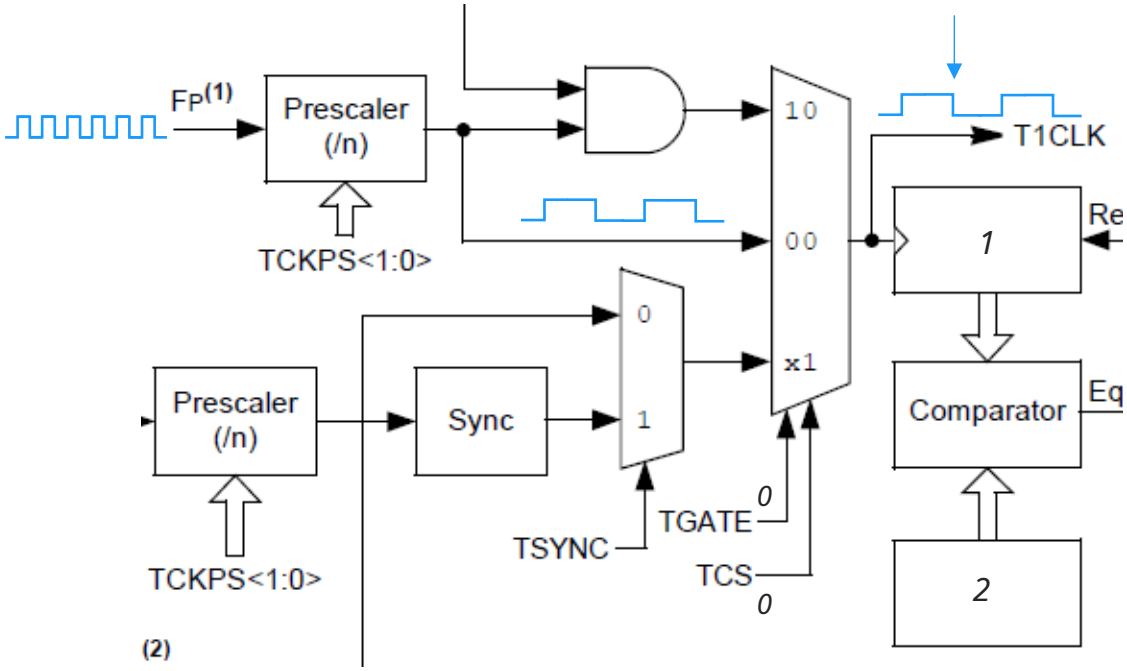
Timer Block Diagram



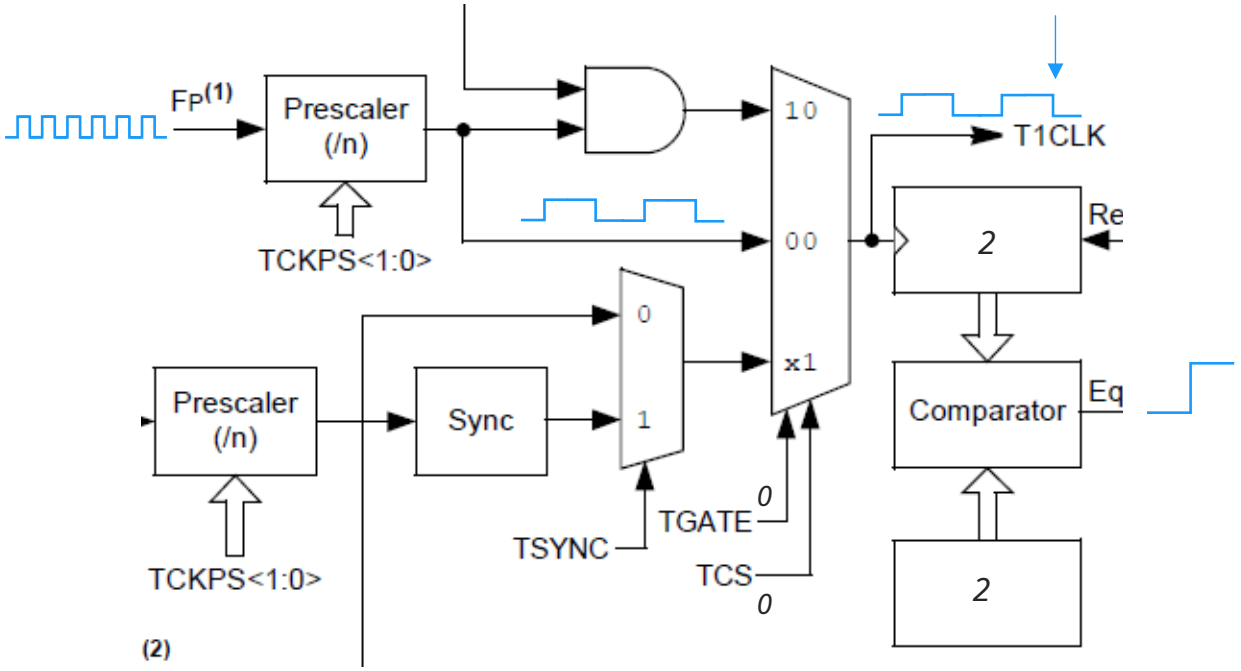
Timer Block Diagram



Timer Block Diagram

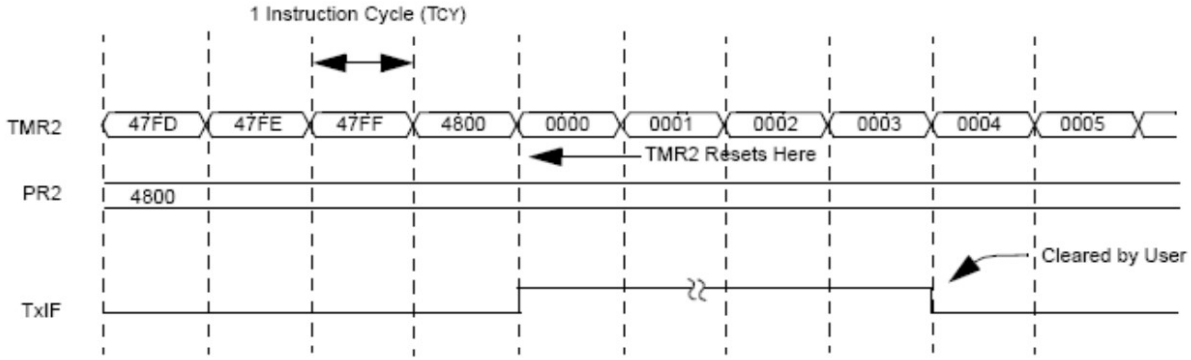


Timer Block Diagram

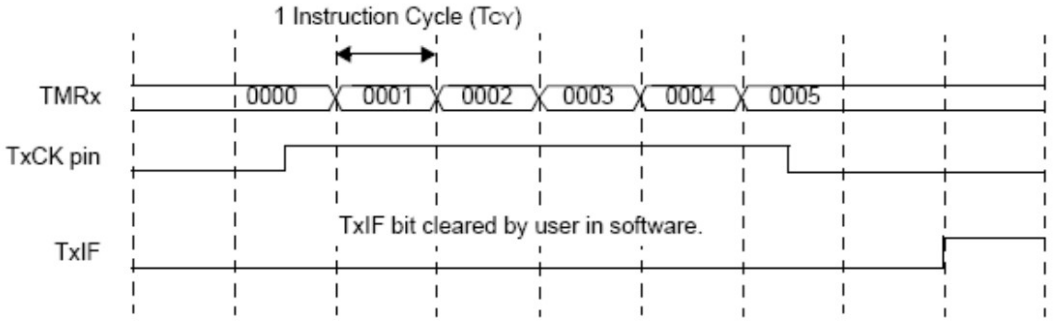


Timer Modes

Counter mode



Gated mode



Timer Registers

Register 12-1: TxCON: Type A Time Base Register

Upper Byte:

R/W-0	U-0	R/W-0	U-0	U-0	U-0	U-0	U-0
TON	—	TSIDL	—	—	—	—	—
bit 15							bit 8

Lower Byte:

U-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0	U-0
—	TGATE	TCKPS<1:0>		—	TSYNC	TCS	—
bit 7							bit 0

TxCON: Timer X Control register.

- TON: starts the timer (set to one after setting up the timer)
- TCKPS: sets the prescaler
- TCS: clock source (0 = internal clock = Fosc/2)
- TGATE: if the gated mode should be enabled

Timer Registers

PRx: set this to the value the timer must count to

TMRx: its the current value of the timer. Set to zero to reset the timer

IFSybits.TxIF: it's the interrupt flag, which signals an event related to the corresponding timer. Must be put to zero manually to be notified of a new event

The interrupt flags are the following ones:

- IFS0bits.T1IF
- IFS0bits.T2IF
- IFS0bits.T3IF
- IFS1bits.T4IF
- IFS1bits.T5IF

Page 58 of the datasheet shows the location of all the IFs

Example Timer Programming

Example: timer expiring every 100 ms, assuming F_{osc} = 8MHz

```
TMR1 = 0; // reset timer counter
// Fcy = Fosc / 2 = 8000000 / 2 = 4000000 (number of clocks in one second)
// in 0.1 second there would be 400000 clocks steps
// this is too high to be put in a 16 bit register (max 65535)
// If we set a prescaler of 1:8 we have 400000/8 = 50000 clock steps, OK!
PR1 = 50000; // (8 MHz / 2) / 8 / 10 < 65535!
T1CONbits.TCKPS = 1; // prescaler 1:8
T1CONbits.TON = 1; // starts the timer!
```

Assignment 1/3

1/ Create two functions inside a file “timer.h” and “timer.c”:

```
#define TIMER1 1
#define TIMER2 2
void tmr_setup_period(int timer, int ms);
void tmr_wait_period(int timer);
```

the first function setups the timer *timer* to count for the specified number of milliseconds. The function should support values up to 200 millisecond. It should start the timer. The second function should use the timer flag to wait until it has expired.

2/ Use the two above defined functions to blink LD1 led at 2.5 Hz frequency (200ms time on, 200ms off)

Template

```
#include "timer.h"
int main() {
    // initialization code
    // [...]
    tmr_setup_period(TIMER1, 200);
    while (1) {
            // code to blink LED
            tmr_wait_period(TIMER1);
    }
}
```

Only one call to tmr_wait_period should be done in any program

Assignment 2/3

1/ Create a function and add it to the timer files:

```
void tmr_wait_ms(int timer, int ms);
```

that uses the timer *timer* to wait for a given number of milliseconds, given as input. The function should setup the timer, wait for it to expire, and then stop the timer. The function should ensure that the timer starts from zero. Note that the timer might have been previously used. The function should support values up to 200 milliseconds.

2/ Use the function to: turn on the led LD1 for 20ms, turn off for 200ms cyclically.

Template

```
#include "timer.h"
int main() {
   // initialization code
   // [...]
   while(1) {
      // [...]
      tmr_wait_ms(TIMER2, 20);
      // [...]
      tmr_wait_ms(TIMER2, 200);
   }
}
```

Assignment 3/3

1/ Modify the tmr_wait_ms() to work with any value of milliseconds.

2/ Modify the tmr_wait_period() to return 1 if the timer is already expired when entering the function, 0 otherwise

3/ Test the code in the next slide and observe the behaviour. Try changing the delay value from 50, 200, 2000.

Template

```
int main() {
    // initialization code
    // [...]
    tmr_setup_period(TIMER1, 200);
    int delay = 50; // 200, 2000
    while (1) {
        tmr_wait_ms(TIMER2, delay);
        // code to blink LD1
        ret = tmr_wait_period(TIMER1);
        if (ret) {
                // turn on LD2
        } else {
                // turn off LD2
        }
    }
}
```

Appendix: Include Guards

What Are Include Guards?

A technique to prevent multiple inclusions of the same header file in a C program.

Avoids "redefinition" errors during compilation

Why Are They Needed?

If a header file is included multiple times (directly or indirectly), the compiler would process its contents repeatedly.

This can cause conflicts, especially with type definitions, constants, and function declarations.

How Do They Work?

```
#ifndef HEADER_FILE_NAME_H
#define HEADER_FILE_NAME_H

// Header file content here

#endif // HEADER_FILE_NAME_H
```

Unique Identifier for each Guard

The define name must be unique in your project!

To ensure so, the name of the file is used typically

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);

#endif // MATH_UTILS_H
```



Università
di Genova